

Patrones Arquitectónicos

A la hora de diseñar o elegir un framework de arquitectura no contamos con una fórmula pero si podemos decir que es un pre-requisito conocer los patrones arquitectónicos para abordar ciertos atributos de calidad. Los principales patrones arquitectónicos son:

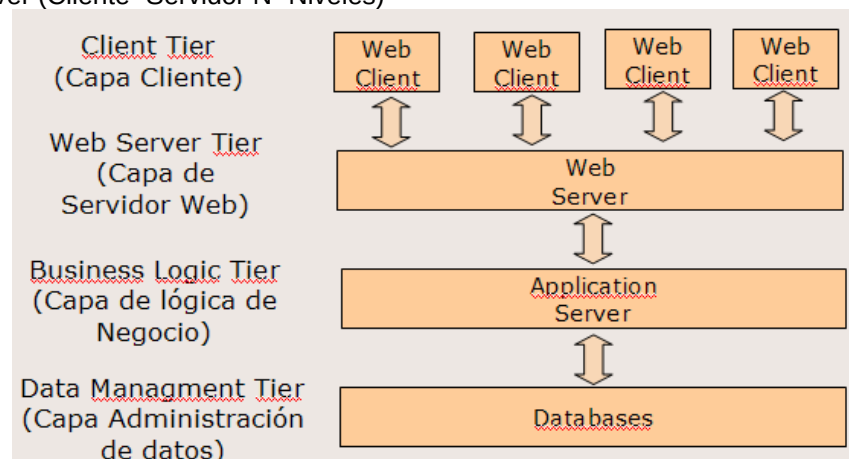
- N – Tier Client- Server (Cliente- Servidor N- Niveles)
- Messaging (Comunicando)
- Publish – Suscribe (Publicar- Suscribir)
- Broker (Agente)
- Process Coordinator (Coordinador de Proceso)

N – Tier Client- Server (Cliente- Servidor N- Niveles)

Las propiedades clave de este patrón son:

- ❖ **Separación de intereses:** La presentación, lógica de negocio y administración de datos están claramente divididos en diferentes capas.
- ❖ **Comunicaciones sincrónicas:** Las comunicaciones entre capas son pedido-respuesta sincrónicas. Las peticiones van en una única dirección desde la capa cliente a través de las capas de servidor Web y lógica de negocios hacia la capa de administración de datos. Cada capa espera la respuesta de la otra capa antes de proceder
- ❖ **Despliegue flexible:** No hay restricciones respecto de cómo las aplicaciones multi-capa se despliegan. Todas las capas podrían correr en la misma máquina o –en el otro extremo- cada capa puede ser desplegada en su propia máquina. En las aplicaciones Web, la capa cliente es usualmente un navegador corriendo en un equipo de escritorio del usuario comunicándose de manera remota vía Internet con los componentes de la capa Web.

Ejemplo: N – Tier Client- Server (Cliente- Servidor N- Niveles)

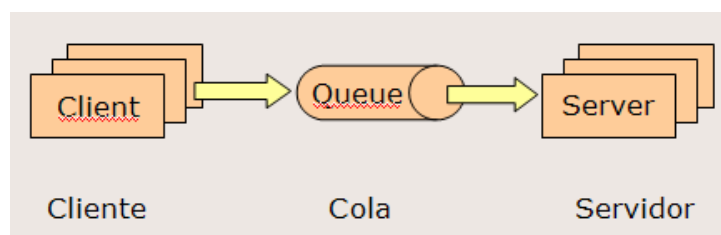


Messaging (Comunicando)

Las propiedades clave de este patrón son:

- ❖ **Comunicaciones asíncronas:** Los clientes envían las peticiones a la cola, donde los mensajes son almacenados hasta que una aplicación los remueve. Después que el cliente ha escrito un mensaje en la cola, continúa trabajando sin esperar que el mensaje sea removido.
- ❖ **Calidad de servicio (QoS) Configurable:** La cola puede ser configurada para una entrega de mayor velocidad, no confiable o entrega de menor velocidad, confiable. Las operaciones de la cola se pueden coordinar con las transacciones de la base de datos.
- ❖ **Bajo Acoplamiento:** No hay vínculo directo entre clientes y servidores. El cliente no es consciente de cuál servidor recibe el mensaje. El servidor no es consciente de qué cliente vino el mensaje.

Componentes del patrón:



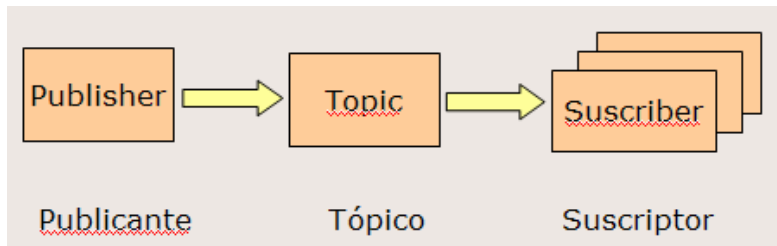
Publish – Suscribe (Publicar- Suscribir)

Las propiedades clave de este patrón son:

- ❖ **Mensajería Muchos-a-Muchos:** Los mensajes publicados son enviados a todos los suscriptores que se han registrado en un tópico. Muchos publicantes pueden publicar sobre el mismo tópico y muchos suscriptores pueden “escuchar” sobre el mismo tópico.

- ❖ **Calidad de servicio (QoS) Configurable:** Además de los mensajes no-confiables y confiables, el mecanismo de comunicación subyacentes puede ser punto-a-punto o multicast/ broadcast. El primero envía un mensaje diferente para cada suscriptor en un tópico, el otro envía un mensaje que cada suscriptor recibe.
- ❖ **Bajo Acoplamiento:** Al igual que “messaging”, no hay vínculo directo entre publicantes y suscriptores. Los publicantes no conocen quién recibe sus mensajes, y los suscriptores no conocen qué publicante envió el mensaje.

Componentes del patrón:

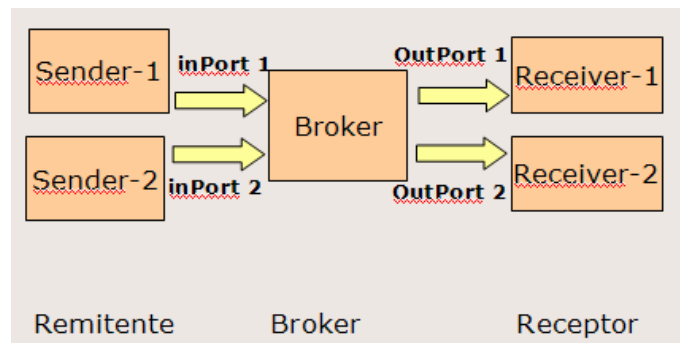


Broker (Agente)

Las propiedades clave de este patrón son:

- ❖ **Arquitectura Hub-and-spoke (Eje-rayo):** El broker actúa como un concentrador de mensajes (eje) y los Remitentes y Receptores se conectan como rayos (como en una rueda de bicicleta) . Las conexiones con el broker son a través de puertos que están asociados con un formato de mensaje específico.
- ❖ **Realiza ruteo de mensaje:** El broker incrusta la lógica de procesamiento para entregar un mensaje recibido en un puerto de entrada en un puerto de salida. El camino de entrega puede estar en el código o depender de valores en el mensaje de entrada.
- ❖ **Realiza transformación del mensaje:** La lógica del broker transforma el tipo de mensaje fuente recibido en el puerto de entrada en el tipo de mensaje de destino requerido en el puerto de salida.

Elementos del patrón:

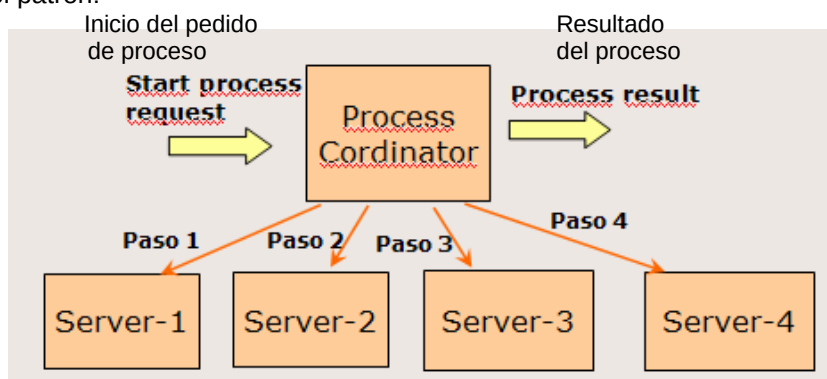


Process Coordinator (Coordinador de Proceso)

Las propiedades clave de este patrón son:

- ❖ **Encapsulamiento de Proceso:** El coordinador de proceso encapsula la secuencia de pasos necesaria para completar un proceso de negocio. La secuencia puede ser arbitrariamente compleja. El coordinador es un único punto de definición para el proceso de negocio, haciendo fácil su comprensión y modificación. Recibe la petición de inicio del proceso, llama a los servidores en el orden definido por el proceso y emite los resultados.
- ❖ **Bajo Acoplamiento:** Los componentes servidores son inconcientes de su rol en todo el proceso de negocio y del orden de los pasos en el proceso. Los servidores simplemente definen un conjunto de servicios que deben realizar y el coordinador los llama como una parte necesaria del proceso de negocio.
- ❖ **Comunicaciones flexibles:** Las comunicaciones entre el coordinador y los servidores pueden ser sincrónicas o asincrónicas. Para comunicaciones sincrónicas el coordinador espera hasta que los servidores responden. Para las comunicaciones asincrónicas el coordinador provee un mecanismo de llamada o respuesta cola/tópico y espera hasta que el servidor responda utilizando el mecanismo definido.

Componentes del patrón:



Comparación a nivel de atributos de calidad.

Patrón Atributo	N – Tier Client- Server	Messaging	Publish – Suscribe	Broker	Process Coordinator
Disponibilidad	Los servidores en cada capa pueden estar replicados, de modo que si uno falla los otros continúan disponibles. En general, la aplicación proveerá una baja calidad de servicio hasta que el servidor que falló sea restaurado.	Las colas físicas con el mismo nombre lógico se pueden replicar a través de diferentes instancias de servidores "messaging". Cuando una falla, los clientes envían los mensajes a la réplica de la cola.	Los tópicos con el mismo nombre lógico pueden ser replicados en diferentes instancias de servidores administrados como un grupo. Cuando uno falla, los publicantes envían los mensajes a las réplicas.	Para construir arquitecturas con alta disponibilidad, los brokers deben estar replicados. Esto es típicamente soportado utilizando mecanismos similares de conjunto de servidores del "messaging" y "publish-subscribe"	El coordinador es un único punto de falla. De ahí que necesite ser replicado para crear una solución con alta disponibilidad.
Manejo de fallas	Si un cliente se está comunicando con el servidor que falla, la mayoría de los servidores y aplicaciones Web implementan transparencia frente a fallos. Esto significa que la petición del cliente, sin que él lo sepa, será re-direccionada a la réplica del servidor que puede satisfacer la petición.	Si un cliente se está comunicando con una cola que falla, puede encontrar una réplica de la cola y colocar el mensaje allí.	Si un publicante se está comunicando con un tópico alojado en un servidor que falla, puede encontrar un servidor réplica y enviar el mensaje allí.	Como los agentes tienen puertos de entrada de tipos definidos, validan y descartan cualquier mensaje enviado en un formato erróneo. Con Brokers replicados, los remitentes pueden sobrellevar una falla dirigiéndose a una réplica del broker en falla.	El manejo de fallas es complejo, puede ocurrir en cualquier etapa de la coordinación del proceso de negocio. Una falla en los últimos pasos en el proceso puede requerir que los primeros pasos sean deshechos usan-o transacciones de compensación. El manejo de fallas necesita un diseño cuidadoso para asegurar que los datos son mantenidos por los servidores restantes de forma consistentes.
Modificabilidad	La separación de intereses favorece la cambiabilidad, dado que la presentación, lógica de negocios y administración de datos están claramente encapsulados. Cada una puede tener modificaciones de su lógica interna, en muchos casos sin cambios en las otras capas.	"Messaging" tiene inherentemente bajo acoplamiento y promueve la alta cambiabilidad ya que clientes y servidores no están directamente sujetos a través de una interfaz. Cambiar el formato de los mensajes enviados por los clientes puede causar cambios en las implementaciones de los servidores. Los formatos de mensajes autodefinidos pueden ayudar a reducir esta dependencia.	Publish-subscribe es inherentemente de bajo acoplamiento lo que promueve la alta cambiabilidad. Nuevos publicantes y suscriptores pueden agregarse al sistema sin cambiar la arquitectura o la configuración. Cambios en el formato de los mensajes publicados pueden causar cambios en las implementaciones de los suscriptores.	Los Brokers separan la lógica de transformación y de ruteo del mensaje desde los remitentes a los receptores. Esto mejora la cambiabilidad ya que cambios en la lógica de ruteo o transformación pueden hacerse sin afectar a los remitentes o receptores.	La cambiabilidad del proceso se favorece porque la definición del proceso está encapsulada en el Coordinador de Procesos. Los servidores pueden cambiar su implementación sin afectar al coordinador o a los otros servidores mientras que la definición del servicio externamente no cambie.
Perfomance	Esta arquitectura provee alto rendimiento. Elementos clave a considerar son la cantidad de hilos concurrentes soportados en cada servidor, la velocidad de las conexiones entre capas y el volumen de transferencia de datos. Como en todos los sistemas distribuidos, es sensato minimizar las llamadas necesarias entre capas para satisfacer cada petición.	La tecnología de cola de mensajes puede entregar miles de mensajes por segundo. Los mensajes menos confiables son más rápidos que los confiables, con la diferencia en la calidad de tecnología de mensajería utilizada.	Puede entregar miles de mensajes por segundo, con mensajes no-confiables más rápido que con los confiables. Si un agente publish-subscribe soporta multicast/broadcast, entregará múltiples mensajes en un tiempo más uniforme a cada suscriptor.	Los Brokers pueden potencialmente convertirse en cuellos de botella especialmente si deben manejar grandes volúmenes de mensajes y ejecutar lógica de transformaciones complejas. Su rendimiento será típicamente más bajo que un "messaging" simple con entrega confiable.	Para proporcionar alto rendimiento el coordinador debe poder manejar múltiples peticiones concurrentes y administrar el estado de cada una a medida que progresan a través del proceso. También, el rendimiento de cualquier proceso está limitado por los pasos más lentos (denominados los servidores lentos en el proceso).

Patrón Atributo	N – Tier Client- Server	Messaging	Publish – Suscribe	Broker	Process Coordinator
Escalabilidad	Como los servidores en cada capa pueden ser replicados y múltiples instancias de servidores corren en el mismo o diferentes equipos servidores, la arquitectura puede escalarse bien. En la práctica la capa de administración de datos frecuentemente se convierte en un cuello de botella en la capacidad de un sistema.	Las colas pueden ser alojadas en los puntos finales de comunicación o ser replicados a través de grupos de servidores de mensajes alojados en un único o en múltiples equipos servidores. Esto lo hace una solución altamente escalable.	Los tópicos pueden estar replicados a través de conjuntos de servidores alojados en un único o en múltiples equipos servidores. Los conjuntos de servidores pueden escalarse para proveer un muy alto volumen de entrega de mensajes. También las soluciones multicast/ broadcast se escalan mejor que sus contrapartes punto-a-punto.	Reunir en clusters instancias de Brokers hace posible construir sistemas escalables para manejar grandes cargas de peticiones. Los Brokers se ubican en aplicaciones en cuyos componentes intercambiar mensajes que requieren una transformación extensa entre la fuente y el destino.	El coordinador puede ser replicado para escalar la aplicación tanto para arriba como para afuera.

Algunas consideraciones generales.

N – Tier Client- Server	Messaging	Publish – Suscribe	Broker	Process Coordinator
Hay que considerar tecnología del servidor de aplicaciones y del servidor web. Es relativamente fácil de implementar. Es adecuado para dar soporte a un número potencialmente grande de clientes y pedidos concurrentes.	Las aplicaciones que pueden dividir el procesamiento de un pedido en pasos discretos, conectados por colas, tienen una extensión de este patrón. Provee una solución resistente para aplicaciones en las cuales la conectividad es transitoria, tanto debido a la red como a la poca confiabilidad de los servidores. Se puede implementar para pedido – respuesta sincrónico utilizando un par de colas pedido – respuesta.	Arquitecturas muy flexibles y adecuadas para aplicaciones que requieren mensajes asincrónicos uno-a-muchos, muchos-a-uno, muchos-a-muchos entre componentes. A igual que el patrón Messaging, la comunicación en ambos lados es posible utilizando pares tópicos pedido – respuesta.	Los agentes son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino. El agente desacopla el remitente del receptor, permitiéndoles producir o consumir su formato nativo, y centraliza la definición de la transformación lógica en el agente para facilitar la comprensión y modificación.	Es utilizado comúnmente para implementar procesos de negocio complejos que deben publicar requerimientos a algunos componentes servidores diferentes. Encapsulando la lógica de proceso en un lugar, es más fácil cambiar, administrar y monitorear la performance del proceso.